



Project Documentation – Speech-to-Text Application (Python Full Stack)

1. Project Overview

The **Speech-to-Text Application** is a web-based project that enables users to convert spoken audio into written text in real-time. It takes voice input through the microphone, processes it via a Speech Recognition API, and displays the recognized text on the screen.

This project is useful for accessibility, transcription, note-taking, and productivity tools.

You'll build it using:

- **Frontend:** [Next.js](#) (or any react framework) + Tailwind CSS
 - **Backend:** Flask or FastApi (Python)
 - **Database :** PostgreSQL / Supabase or Convex
 - **Speech Recognition API:** DeepInfra API or Google Speech-to-Text or Deepgram (Free)
-

2. Aim of the Project

- Create a functional web application that captures user speech and converts it into accurate, readable text.
- Learn and apply industry-relevant technologies like **Next.js (frontend)**, **Flask (backend)**, and **API integration**.
- Gain hands-on experience with **real-time applications** and improve understanding of **frontend-backend interaction**.
- Strengthen deployment knowledge for both frontend and backend.

3. Skills to Learn from this Project

◆ Frontend Development

- Next.js (pages, API routes, rendering methods)
- Tailwind CSS for building responsive UI
- Browser APIs for microphone access (`navigator.mediaDevices`)
- Real-time updates with **WebSockets (Flask-SocketIO)** (optional advanced feature)

◆ Backend Development (Python + Flask)

- Flask basics: routes, request handling, and responses
- Creating REST APIs for audio upload & speech processing
- Handling multipart/form-data for audio files
- Using Python libraries like `pydub` or `speechrecognition` (local fallback)
- Flask-SocketIO for streaming real-time transcription

◆ API Integration

- Using DeepInfra or Google Speech-to-Text API
- Managing authentication keys securely (dotenv or Flask config)
- Parsing API responses into user-friendly text

◆ Deployment & Hosting

- Deploying **frontend** on Vercel/Netlify
- Deploying **Flask backend** on Render/Heroku

- Managing CORS between frontend and backend

♦ **Soft Skills**

- Planning and scheduling with a **2-week development roadmap**
 - Writing professional technical documentation
 - Debugging full-stack applications
-

4. Use Cases

The **Speech-to-Text Application** can be applied in several real-world scenarios:

- **Accessibility tools** – Helping people with hearing/speaking disabilities.
 - **Meeting or lecture transcription** – Record and convert conversations into text.
 - **Voice commands** – Base for smart assistants and automation apps.
 - **Hands-free writing** – Dictation for writers, bloggers, or coders.
 - **Language learning** – Assisting students in practicing pronunciation.
-

5. Tech Stack (Python Full Stack)

Frontend (Client-Side)

- Next.js
- Tailwind CSS
- Axios / Fetch API (for API calls)

Backend (Server-Side)

- Flask (Python)
- Flask-CORS (for frontend-backend communication)
- Flask-SocketIO (optional, for real-time streaming text)

Database

- PostgreSQL / Supabase or Convex

APIs / Libraries

- DeepInfra Speech-to-Text API or Google Cloud Speech API
- Python Libraries: `speechrecognition`, `pydub`, `requests`, `flask`, `flask-cors`

Deployment

- Frontend → Vercel / Netlify
- Backend → Render / Heroku

6.Day-by-day plan

Day 1 — Project scaffolding & planning

Goals: repo, branches, wireframe, environment setup.

- Create repo and folders: `frontend/`, `backend/`.
- Create `README.md`, `.gitignore`, `.env.example`, `LICENSE`.
- Create branches: `main` (stable) and `dev` (work).
- Quick wireframe: sketch 2 screens — (1) Recorder + live transcript, (2) Transcript history.

- Set up frontend baseline: `npx create-next-app@latest frontend` (or `app router` if you prefer).
- Create Python virtualenv for backend: `python -m venv venv` and activate.
- Create `backend/requirements.txt` with `flask`, `flask-cors`, `pydub`, `requests`, `flask-socketio` (optional), `gunicorn`.
- Create `frontend/README` and `backend/README` skeletons.

Deliverable: repo with folders, initial commits, and wireframes in README.

Day 2 — Frontend UI skeleton & Tailwind

Goals: basic UI and layout, Tailwind installed.

- Install Tailwind: inside `frontend/` follow Tailwind setup (init `tailwind.config.js`, add `@tailwind base;` etc.).
- Create UI components: `Header`, `RecorderPanel`, `TranscriptPanel`.
- Build static pages: `/` (recorder) and `/history`.
- Add placeholder transcript area and start/stop buttons.
- Add a simple store/state for transcript text (React state or context).

Deliverable: clickable UI mockup with start/stop buttons and empty transcript area.

Day 3 — Client-side(Frontend) audio capture (MediaRecorder)

Goals: capture microphone audio and save to Blob.

- Implement microphone permission request using `navigator.mediaDevices.getUserMedia({ audio: true })`.
- Use `MediaRecorder` to record audio chunks, assemble into a single `Blob` on stop.
- Show recording state (recording icon, timer).
- Provide `Download .wav` button to verify recorded audio locally.

Small code snippet (frontend)

// simplified — run in component

```
const stream = await navigator.mediaDevices.getUserMedia({
  audio: true });
```

```
const mediaRecorder = new MediaRecorder(stream);
```

```
const chunks = [];
```

```
mediaRecorder.ondataavailable = e => chunks.push(e.data);
```

```
mediaRecorder.onstop = () => {  
  
  const blob = new Blob(chunks, { type: 'audio/webm' });  
  
  // convert/send blob  
  
};  
  
mediaRecorder.start();
```

Deliverable: working record/stop and saved Blob (able to download & play locally).

Day 4 — Flask backend: basic upload endpoint

Goals: accept audio uploads and return a placeholder response.

- Create Flask app skeleton with CORS enabled (`flask-cors`).
- Implement `/transcribe` POST endpoint that accepts multipart form `file` and saves temp file.
- Return JSON `{ status: 'ok', message: 'received', filename }` to confirm roundtrip.

Small code snippet (backend)

```
from flask import Flask, request, jsonify  
  
from flask_cors import CORS
```

```
app = Flask(__name__)
```

```
CORS(app)
```

```
@app.route('/transcribe', methods=['POST'])
```

```
def transcribe():
```

```
    f = request.files.get('file')
```

```
    if not f:
```

```
        return jsonify({'error': 'no file'}), 400
```

```
    path = f"/tmp/{f.filename}"
```

```
    f.save(path)
```

```
    return jsonify({'status': 'ok', 'path': path})
```

Deliverable: backend receives file; test by curl or Postman.

Day 5 — Integrate Speech-to-Text provider (proof of concept)

Goals: get a working transcript from an STT provider.

- Choose provider: DeepInfra or Google Cloud Speech.
- Add API key / credentials to **backend/.env** (keep secret).

- Implement a function to call STT API with the saved audio file.
- Return the transcript in the `/transcribe` response.

Notes & tips:

- With Google: you may upload audio to GCS or send bytes directly (depends on API).
- With DeepInfra: usually a REST call with audio bytes; check provider docs for required format and sample rate.
- If needed, use `pydub` to convert audio to a uniform WAV (16kHz mono) before sending.

Deliverable: `/transcribe` returns real transcript text for uploaded audio.

Day 6 — Wire frontend to backend (end-to-end)

Goals: frontend sends recorded Blob to backend and displays transcript.

- Convert Blob to `File` and send via `fetch` or `axios` using `FormData`.
- Show a loading state while waiting for backend.
- Display returned transcript in the transcript panel.

Frontend upload example

```
const form = new FormData();
```

```
form.append('file', recordedFile, 'speech.webm');

const res = await fetch(process.env.NEXT_PUBLIC_API +
'/transcribe', {

method: 'POST',

body: form

});

const data = await res.json();

setTranscript(data.transcript);
```

Deliverable: working end-to-end transcription flow in dev mode.

Day 7 — Improve audio handling & format compatibility

Goals: ensure audio is compatible with STT and handle bigger recordings.

- Use **pydub** on server to convert to WAV/16k mono if needed:
`AudioSegment.from_file(path).set_frame_rate(16000).set_channels(1)` then export.
- Add file size guard and streaming/chunking strategy if files are large.

- Add server-side validations and clear error messages.

Deliverable: robust audio conversions and fewer failed transcriptions.

Day 8 — (Optional) Real-time streaming / partial transcripts(Chunking)

Goals: implement streaming for lower latency (advanced).

- Add `flask-socketio` to backend and socket client to frontend.
- Send small audio chunks to server via WebSocket; server forwards to STT streaming endpoint (if provider supports streaming) and emits partial transcripts back to client.
- If provider doesn't support streaming, implement small-chunk buffering and send frequent short requests.

Deliverable: (Optional) partial transcript live on screen during recording.

Day 9 — Persist transcripts & build history UI

Goals: save transcripts to DB and show history.

- Choose DB: Postgres or Supabase.

- Schema suggestion: `transcripts(id, user_id nullable, text, created_at, duration_seconds, filename, language)`
- Implement endpoints: `GET /transcripts` and `GET /transcripts/:id`.
- Build `/history` UI to list transcripts and open/download them.

Deliverable: transcripts persist and are viewable in history page.

Day 10 — Authentication & multi-user support

Goals: allow users to sign in and save private transcripts.

- Simple route: use Supabase Auth (fast) or implement JWT with Flask (PyJWT).
- Secure upload endpoint to attach `user_id`.
- Protect `/history` for authenticated users.

Deliverable: basic auth & private transcript storage.

Day 11 — UI polish & export features

Goals: make app pleasant and add export options.

- Add transcript utilities: `Copy`, `Download .txt`, `Download .docx` (optional using `docx` library), `Share link`.
- Improve responsive design and accessibility (aria-labels, keyboard shortcuts).
- Add timestamps or speaker labels (if implemented upstream).

Deliverable: polished UI and download features.

Day 12 — Testing & QA

Goals: test on multiple browsers and fix bugs.

- Cross-browser test: Chrome, Firefox, Edge, mobile Safari.
- Add unit tests for backend with `pytest` + Flask test client.
- Manual E2E: record short clips, long clips, noisy clips.
- Test file size limits, concurrency, and error flows.

Deliverable: test notes and a list of fixed bugs.

Day 13 — Deployment

Goals: make production-ready and deploy.

- Backend: create `Procfile` or define start command:
`gunicorn app:app --bind 0.0.0.0:$PORT`.
- Frontend: set `NEXT_PUBLIC_API` env var to production backend URL.
- Set environment variables in hosting (STT API key, DB URL).
- Optional: add simple CI with GitHub Actions to run tests and build.
- Run a final end-to-end test on staging.

Deliverable: deployed staging version and documented deployment steps.

Day 14 — Final polish, docs & demo

Goals: finalize README, create demo, and plan next steps.

- Write final `README.md` with setup, run, deploy instructions and architecture diagram.
- Create a short demo video or GIF showing: record → upload → transcript → download.
- Prepare a short changelog and TODOs for future enhancements (speaker diarization, punctuation, summarization, translation).
- Merge `dev` into `main` and create a release tag.

Deliverable: production app, README, demo, and release tag.

7. Resources

♦ YouTube Tutorials

- [Next.js Crash Course – Traversy Media](#)
- [Tailwind CSS Crash Course – Traversy Media](#)
- [Node.js & Express Crash Course – Traversy Media](#)
- [Flask Crash Course – Tech With Tim](#)
- [Speech Recognition with JavaScript – The Net Ninja](#)
- <https://deepgram.com/learn/best-speech-to-text-apis>
- <https://deepinfra.com/>

♦ API Documentation

- DeepInfra Speech-to-Text API Docs
- Google Speech-to-Text Docs

♦ Extra Learning

- Deploying Next.js Apps to Vercel
- Render Deployment Guide for Node/Flask